

Lec01 - Algorithms, Pseudocode, Analysis

Algorithm = step-by-step procedure to solve a problem; input -> output; clear and finite.

Property	Meaning
Correct	right output
Unambiguous	clear steps
General	works all cases
Simple	easy to understand
Finite	must terminate

Applications: data retrieval, network routing, sorting, searching, shortest path.

Analysis predicts resource usage: time, memory, logic gates. Cases: best case = minimum steps; worst case = maximum steps; average case = average steps. RAM model: one processor, one instruction at a time; assignment/arithmetic/comparison/memory access counted as constant steps.

Input

Algorithm

Output

Lec02 - Insertion Sort, Big O

Insertion sort builds a sorted part one item at a time; key is shifted left into correct position.

INSERTION-SORT (A)
for j = 2 to A.length
 key = A[j]
 i = j - 1
 while i > 0 and A[i] > key
 A[i+1] = A[i]
 i = i - 1
 A[i+1] = key

Runtime patterns
single loop: O(n)
nested n^m: O(n^m^2)
i!=i^2: O(log n)
n + n^2: O(n^2)

Case	Insertion	Bubble	Selection
Best	O(n)	O(n)*	O(n^2)
Average	O(n^2)	O(n^2)	O(n^2)
Worst	O(n^2)	O(n^2)	O(n^2)

Asymptotic notation focuses on growth for large n. O = upper bound; Omega = lower bound; Theta = tight bound. Growth order: 1 < log n < n < n log n < n^2 < n^3 < 2^n n. Biggest term only: 3n^2 + 5n + 7 -> O(n^2). Separate loops add; nested loops multiply; constants are ignored.

Lec03 - Stack

Stack: access only last inserted item. LIFO = Last In First Out.

Operation	Meaning	Time
push(x)	insert at top	O(1)
pop()	remove top	O(1)
peek()	read top only	O(1)
isEmpty	top == -1	O(1)
isFull	top == maxSize-1	O(1)

push(x): if !isFull -> stack[++top]=x
pop(): if !isEmpty -> return stack[top--]
peek(): return stack[top]

Applications: string reversal, browser history, undo, recursive calls, algorithm auxiliary structure, memory stack for process/method calls.

Top

30

20

10

LIFO

Lec04 - Queue and Circular Queue

Queue: FIFO = First In First Out. Insert at rear; remove from front.

Operation	Meaning
insert(x)	add at rear
remove()	remove front
peekFront()	read front only
isEmpty	nItems == 0
isFull	nItems == maxSize

Circular queue/ring buffer reuses freed spaces. rear=(rear+1)%maxSize after insert; front=(front+1)%maxSize after remove. Linear queue can waste front spaces when rear reaches the end.

Front/remove

10

20

30

40

Rear/insert

FIFO

Lec05 - Linked Lists

Linked list: nodes connected by references. Each node has data and next. Last node points to null.

first

10

20

30

null

Array	Linked List
direct index access	sequential access
fixed size	dynamic size
continuous memory	non-continuous memory

class Link { int data; Link next; }
isEmpty(): first == null
insertFirst(x):
 newLink.next = first
 first = newLink
find(key): move current=current.next
insertAfter:
 newLink.next = current.next
 current.next = newLink
delete first: first = first.next
delete middle/last: previous.next = current.next
Applications: image viewer previous/next, browser back/next, music player, implement stacks/queues, adjacency list graph representation, dynamic memory allocation, directory of names. Doubly linked list: each node has next and previous; supports forward/backward traversal.

Lec06 - Trees and BST

Tree: collection of nodes connected by edges. Node = data item; edge = relationship. Binary tree = each node has at most two children. Root = top node; parent = node with child; child = node below parent; leaf = no children; path = connected route; subtree = node with descendants; key value = chosen data item used for searching.

50

30

70

20

40

60

80

Binary Search Tree (BST)

Rule	Meaning
left child key < parent	go left
right child key >= parent	go right
minimum	keep going left
maximum	keep going right

find(key): compare from root
if key < current.iData -> leftChild
else -> rightChild
insert(id): compare until empty place;
link new node as left or right child.
BFS = breadth first search, level by level. DFS = depth first search: inorder, preorder, postorder.

Lec07 - Traversals, Delete, Tree Types

Traversals orders

Traversal	Order	Root position
Inorder	Left-Root-Right	middle
Preorder	Root-Left-Right	first
Postorder	Left-Right-Root	last

BST inorder: 20,30,40,50,70
Preorder: 50,30,20,40,70
Postorder: 20,40,30,70,50

10

5

50

25

75

one child: link past node's children: use successor

Delete cases: (1) no child: remove link from parent. (2) one child: parent points to deleted node's child. (3) two children: replace deleted node using inorder successor, usually the minimum node in right subtree.

Term	Formula/Meaning
Degree	number of children
Depth	path length from root
Height	maximum depth
Full binary tree	nodes = 2^((h+1)-1)
Complete binary tree	filled left to right
Complete height	floor(log2 n)

Core Java Pattern - Stack and Queue

```
class StackX {  
  private int maxSize, top;  
  private double[] stackArray;  
  StackX(int s){ maxSize=s; stackArray=new double[s]; top=-1; }  
  void push(double x){ if(!isFull()) stackArray[++top]=x; }  
  double pop(){ if(isEmpty()) return -99; return stackArray[top--]; }  
  double peek(){ if(isEmpty()) return -99; return stackArray[top]; }  
  boolean isEmpty(){ return top == -1; }  
  boolean isFull(){ return top == maxSize-1; }  
}  
  
class QueueX {  
  private int maxSize, front, rear, nItems; private int[] q;  
  QueueX(int s){ maxSize=s; q=new int[s]; front=0; rear=-1; nItems=0; }  
  void insert(int x){ if(!isFull()){ rear=(rear+1)%maxSize; q[rear]=x; nItems++; } }  
  int remove(){ if(isEmpty()) return -99; int t=q[front]; front=(front+1)%maxSize; nItems--; return t; }  
  int peekFront(){ if(isEmpty()) return -99; return q[front]; }  
  boolean isEmpty(){ return nItems==0; }  
  boolean isFull(){ return nItems==maxSize; }  
}
```

Core Java Pattern - Linked List

```
class Link { int data; Link next; Link(int d){ data=d; next=null; } }  
class LinkList {  
  private Link first;  
  LinkList(){ first=null; }  
  boolean isEmpty(){ return first==null; }  
  void insertFirst(int d){ Link n=new Link(d); n.next=first; first=n; }  
  Link deleteFirst(){ if(isEmpty()) return null; Link t=first; first=first.next; return t; }  
  Link find(int key){  
    Link cur=first;  
    while(cur!=null){ if(cur.data==key) return cur; cur=cur.next; }  
    return null;  
  }  
  boolean insertAfter(int key,int d){  
    Link cur=first;  
    while(cur!=null){  
      if(cur.data==key){ Link n=new Link(d); n.next=cur.next; cur.next=n; return true; }  
      cur=cur.next;  
    }  
    return false;  
  }  
  boolean delete(int key){  
    Link cur=first, prev=first;  
    while(cur!=null){  
      if(cur.data==key){ if(cur==first) first=first.next; else prev.next=cur.next; return true; }  
      prev=cur; cur=cur.next;  
    }  
    return false;  
  }  
}
```

Core Java Pattern - BST Traversals

```
class Node { int iData; double dData; Node leftChild, rightChild; }  
class Tree { private Node root;  
  Node find(int key){  
    Node cur=root;  
    while(cur!=null && cur.iData!=key)  
      cur = (key < cur.iData) ? cur.leftChild : cur.rightChild;  
    return cur;  
  }  
  void insert(int id,double dd){  
    Node n=new Node(); n.iData=id; n.dData=dd;  
    if(root==null){ root=n; return; }  
    Node cur=root, parent;  
    while(true){ parent=cur;  
      if(id < cur.iData){ cur=cur.leftChild; if(cur==null){ parent.leftChild=n; return; } }  
      else { cur=cur.rightChild; if(cur==null){ parent.rightChild=n; return; } }  
    }  
  }  
  void inOrder(Node r){ if(r!=null){ inOrder(r.leftChild); display(r); inOrder(r.rightChild); } }  
  void preOrder(Node r){ if(r!=null){ display(r); preOrder(r.leftChild); preOrder(r.rightChild); } }  
  void postOrder(Node r){ if(r!=null){ postOrder(r.leftChild); postOrder(r.rightChild); display(r); } }  
}
```

Page 1 / 2

Lec08 - Recursion and Recurrence

Recursion: a function calls itself to solve a smaller version of a problem. It must have a base case/final case to stop. Recursive case calls itself.

Function

Base case

Recursive relation

factorial

fact(0)=1

fact(n)=n*fact(n-1)

sum

sum(0)=0

sum(n)=n+sum(n-1)

power

pow(x,0)=1

pow(x,n)=x*pow(x,n-1)

fact(4)

4*fact(3)

act(2)

fact(1)

1*fact(0)

1

values return up

fact(n): if n==0 return 1 else return n*fact(n-1)
sum(n): if n==0 return 0 else return n+sum(n-1)
pow(x,n): if n==0 return 1 else return x*pow(x,n-1)
Recurrence equation expresses running time using smaller input. One call with n-1: T(n)=T(n-1)+O(1). One call with n/2: T(n)=T(n/2)+O(1). Two calls with n/2: T(n)=2T(n/2)+O(1).

Master Method: $T(n)=aT(n/b)+f(n)$

Compare f(n) with n^c (log_b a)	Result
smaller by polynomial	$\Theta(n^c \log_b a)$
same order	$\Theta(n^c \log_b a \log n)$
larger + regularity	$\Theta(f(n))$

Lec09 - Divide and Conquer, Quick Sort

Divide and Conquer: divide problem into subproblems, conquer/solve subproblems, combine solutions. Applications: Quick Sort and Merge Sort. Quick Sort does more work in divide/partition; combine work is minimal because it sorts in place.

1025430020

10,4,0 pivot [25,30]

QUICKSORT(A, p, r)
if p < r
 q = PARTITION(A, p, r)
 QUICKSORT(A, p, q)
 QUICKSORT(A, q+1, r)

PARTITION: choose pivot; move <= pivot left, >= pivot right.

Case	Partition	Recurrence	Time
Best	balanced	$T(n)=2T(n/2)+\Theta(n)$	$\Theta(n \log n)$
Average	near balanced	-	$O(n \log n)$
Worst	0 and n-1	$T(n)=T(n-1)+\Theta(n)$	$\Theta(n^2)$

Lec10 - Merge Sort

Merge Sort divides array into two halves, recursively sorts each half, and merges two sorted halves. It has better worst-case growth than insertion sort. Main combine work is merging.

[6,4,8,1]

[6,4]

[8,1]

[4,6]

[1,8]

[1,4,6,8]

MERGESORT(A, p, r)
if p < r
 q = floor((p+r)/2)
 MERGESORT(A, p, q)
 MERGESORT(A, q+1, r)
 MERGE(A, p, q, r)

MERGE uses temporary arrays L and R with infinity sentinels. Compare L[i] and R[j]; copy smaller to A[k].
Analysis: $T(n)=2T(n/2)+\Theta(n) = O(n \log n)$. Temp array/temporary arrays hold values while two sorted parts are combined.

Lec11 - Heaps and Heap Algorithms

Heap is an array represented as a complete binary tree. Complete binary tree: all levels filled except possibly last; last level filled left to right. Max heap: parent >= children. Min heap: parent <= children. Root is A[1].

90

5070

203060

Array: [90,50,70,20,30,60]

Index formula (1-based)	Value
PARENT(i)	floor(i/2)
LEFT(i)	2i
RIGHT(i)	2i+1
height of complete tree	floor(log2 n)

MAX_HEAPIFY(A, i)
l = LEFT(i); r = RIGHT(i)
if l <= heap_size and A[l] > A[i]
 largest = l
else largest = i
if r <= heap_size and A[r] > A[largest]
 largest = r
if largest != i
 exchange A[i] with A[largest]
 MAX_HEAPIFY(A, largest)
MAX_HEAPIFY maintains max-heap property by moving the largest of parent/left/right to the parent position and continuing downward. Running time: $O(\log n)$, because the call can move down the height of heap.

Lec12 - Build Heap, Heapsort, Priority Queue

BUILD_MAX_HEAP converts an unsorted array into a max heap. Start from last non-leaf node floor(n/2), heapify bottom-up until root. Complexity: $O(n)$.

BUILD_MAX_HEAP(A)
A.heap_size = A.length
for i = floor(A.length/2) downto 1
 MAX_HEAPIFY(A, i)

HEAPSORT(A)
BUILD_MAX_HEAP(A)
for i = A.length downto 2
 exchange A[i] with A[1]
 A.heap_size = A.heap_size - 1
 MAX_HEAPIFY(A, i)

Heapsort uses max heap for ascending order. Largest value at root A[1] is swapped with last item; heap size reduces; MAX_HEAPIFY fixes root. Complexity: $O(n \log n)$. In-place sorting.

HEAP_EXTRACT_MAX(A)
if heap_size < 1 return error
max = A[1]
A[1] = A[heap_size]
heap_size = heap_size - 1
MAX_HEAPIFY(A, 1)
return max

Practical Sorts - Bubble and Selection

Bubble Sort: compare adjacent elements and swap if left value is greater than right value. After each pass, largest unsorted value moves to the end. In-place comparison sort.

BUBBLE-SORT(A)
for i = 0 to n-2
 swapped = false
 for j = 0 to n-i-2
 if A[j] > A[j+1]
 swap A[j], A[j+1]; swapped = true
 if swapped == false break

Selection Sort: sorted part at beginning; repeatedly select minimum from unsorted part and swap with current position.

SELECTION-SORT(A)
for i = 0 to n-2
 min = i
 for j = i+1 to n-1
 if A[j] < A[min] then min = j
 swap A[i], A[min]

Complexity Reference

Structure / Algorithm	Main operation	Time
Stack push/pop/peek	top index	$O(1)$
Queue insert/remove	front/rear	$O(1)$
Linked list find	sequential scan	$O(n)$
BST find/insert average	height h	$O(h)$ avg $O(\log n)$
BST worst	skewed tree	$O(n)$
MAX_HEAPIFY	height	$O(\log n)$
BUILD_MAX_HEAP	bottom-up heapify	$O(n)$
HEAPSORT	repeated heapify	$O(n \log n)$

Algorithm	Best	Average	Worst	Space
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Bubble*	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ avg
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

Pseudocode and Runtime Patterns

Pseudocode uses clear English-like steps, indentation for loops/conditions, assignment statements, and comments. Input and output can be stated before algorithm body.

MAX-OF-THREE(a, b, c)
max = a
if b > max then max = b
if c > max then max = c
return max

AVERAGE(A, n)
sum = 0
for i = 0 to n-1
 sum = sum + A[i]
return sum / n

Code pattern	Big O
constant statements	$O(1)$
for i=1 to n	$O(n)$
two separate n loops	$O(n)$
nested n by n loops	$O(n^2)$
i=12 or n=n/2	$O(\log n)$
n work per log n levels	$O(n \log n)$

Line-Level Meanings

Line / Symbol	Meaning
top = -1	stack empty
top == maxSize-1	stack full
stack[++top] = x	increase top, then insert
stack[top--]	return top, then decrease top
front	queue remove position
rear	queue insert position
nItems	current queue item count
(index+1)%maxSize	circular movement
first == null	linked list empty
current = current.next	move to next linked-list node
newLink.next = current.next	new node points to old next
current.next = newLink	current points to new node
previous.next = current.next	skip/delete current node
leftChild/rightChild	BST navigation
A[1]	heap root

Small Visual Rules

Stack

Top

30

20

10

Queue

Front/remove

10

20

30

40

Rear/insert

LIFO

10

20

30

null

FIFO

10

20

30

null

Heap: parent >= children (max)